Part Four - Play-test, and show it

Today somebody else plays your game.

7.6 Play-testing

Spend the first half of the meeting on what your play-testers found. Watch them play, with your hands off the keyboard. Whatever confuses them is a real problem. Go fix it, then watch somebody play it again.

That cycle, build, test, change, test again, is what **play-testing** is.

7.7 Show it

Play a neighbor's game. Tell them one thing you liked, and one thing you would add.

Then be ready to say two things about your own game: what it is, in one sentence, and one decision you made on purpose.

Chapter summary

You started with a game that could end, once. You end with one that can be won, lost, and played again, that's yours from the character to the rules, and that somebody else has played.

You built	It's an example of
`playerWon`, and three screens	three **modes**, and `draw()` picking one
`!gameOver && !playerWon`	`&&`, **and**
`else if`	a list of questions, and the first true one wins
`restartGame()`, called from `setup()`	**restarting** by running the starting steps again
`if (gameOver || playerWon)` in `mousePressed()`	`||`, **or**, guarding the restart
`goal = 1`	testing by cheating, again
three coins, three `if`s	repetitive and readable: a trade-off
your neighbor, playing	**play-testing**

New syntax in this chapter

Syntax	Where you met it
`&&`	7.3
`else if`	7.3
`restartGame()`	7.4
`||`	7.4
`Sprite.screenBehavior = ScreenBehavior.BouncyWalls;`	Step 7

Vocabulary

game loop -- `draw()` running over and over. It's the engine of every game you've ever played. (7.2)

play-testing -- playing your own game, or watching somebody else play it, to find out whether it's fun, and then changing it. (7.6)

restart -- putting every variable back to its starting value, so the game can be played again. (7.4)

win / lose condition -- the rules that decide when the game ends, and how. (7.3)

Check your understanding

1. What does `restartGame()` do, and why does `setup()` call it?
2. Why are the ground and clouds built in `setup()` and not in `restartGame()`?
3. What does `!gameOver && !playerWon` mean, in plain words?
4. Why does `mousePressed()` check `gameOver || playerWon` before it restarts?
5. You restart, and the score goes back to `0`, and the game stays on the game over screen. What's missing?
6. What's your game, in one sentence? What's one decision you made on purpose?

What's next

That's the course. Seven chapters ago there was an empty window, and now there's a game in it that somebody else has played.

If you want more, the bonus chapters are at the back: pictures, sounds, animation, and lists. Next quarter, the code leaves the screen, with Arduino and the Circuit Playground board.